

Trivial Flag Transfer Protocol

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Figure out how they moved the flag.

Hints

- What are some other ways to hide data?
-

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - **Wireshark** – GUI tool for analyzing network traffic (TFTP object extraction)
 - `tr` – ROT13 decoding
 - `zsteg` – Detect steganography in BMP images
 - `steghide` – Extract embedded data from BMP files
-

Complete Solution

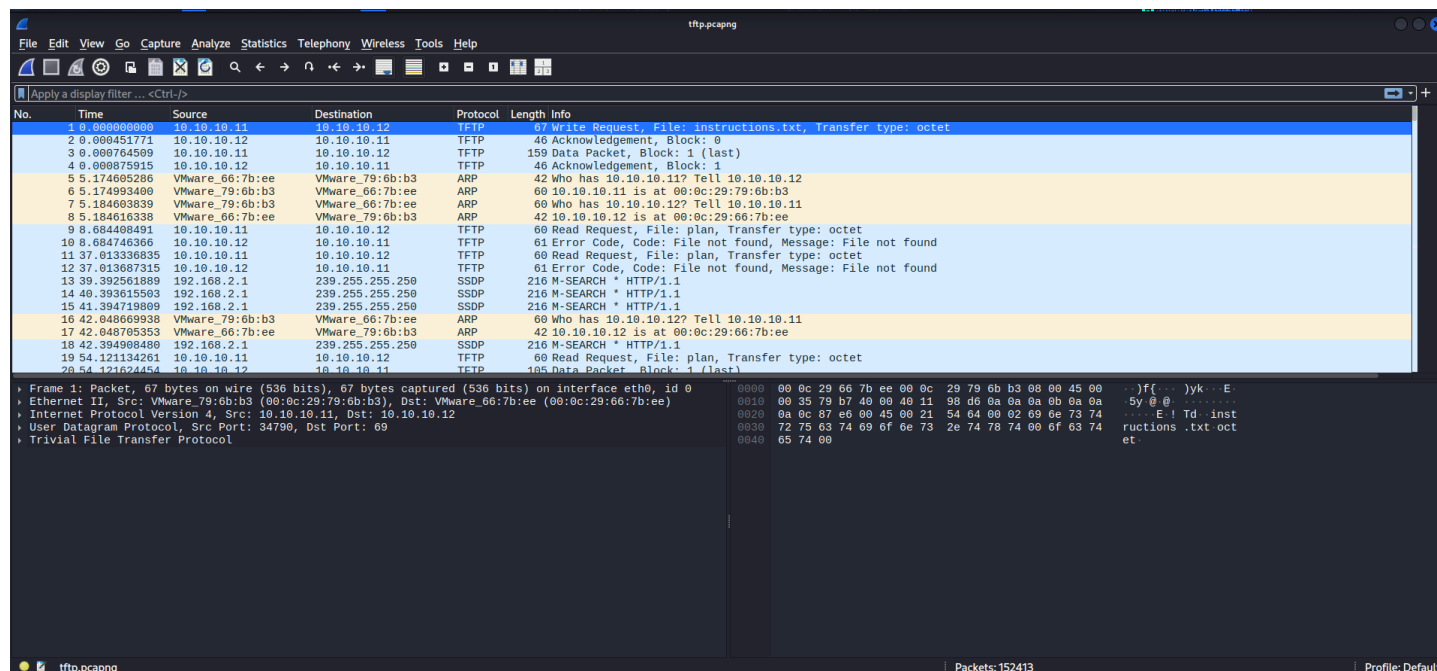
Step 1: Download and Prepare

Download the PCAP file named `tftp.pcapng` from the challenge page.

Nothing suspicious.

Step 5: Analyze with Wireshark

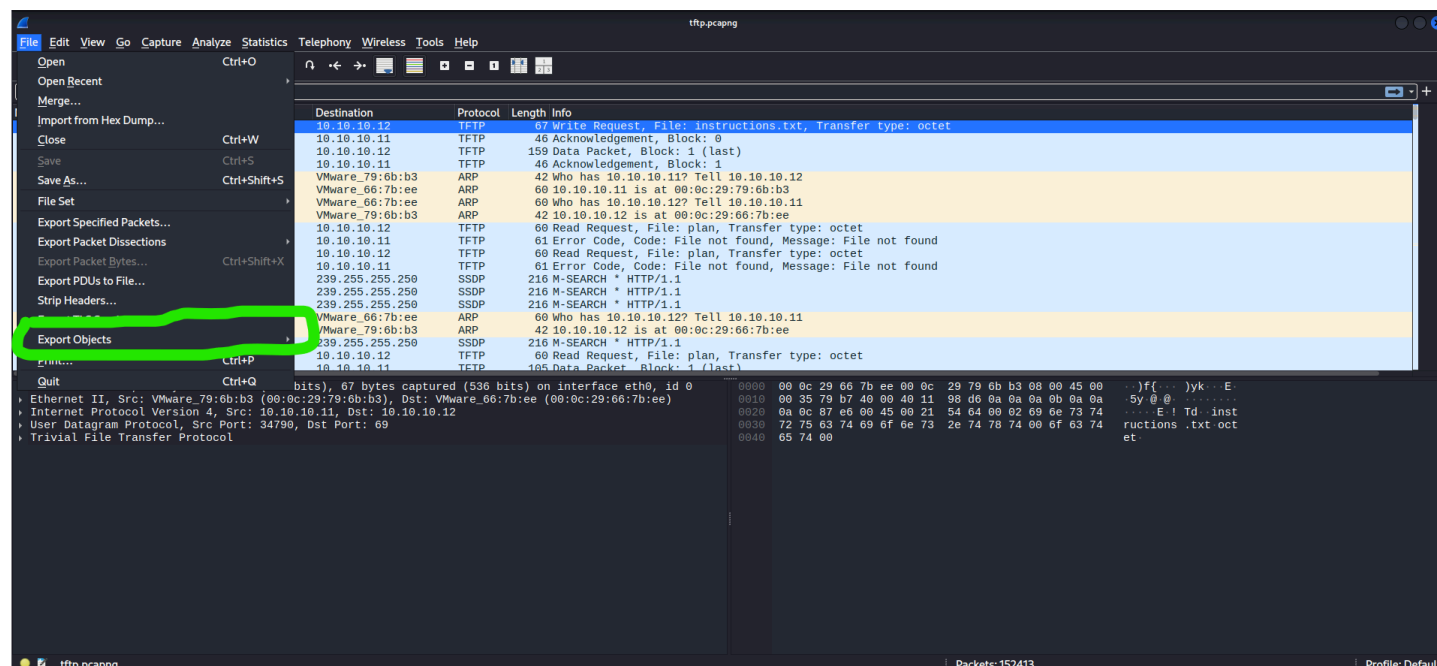
Open the file in Wireshark:



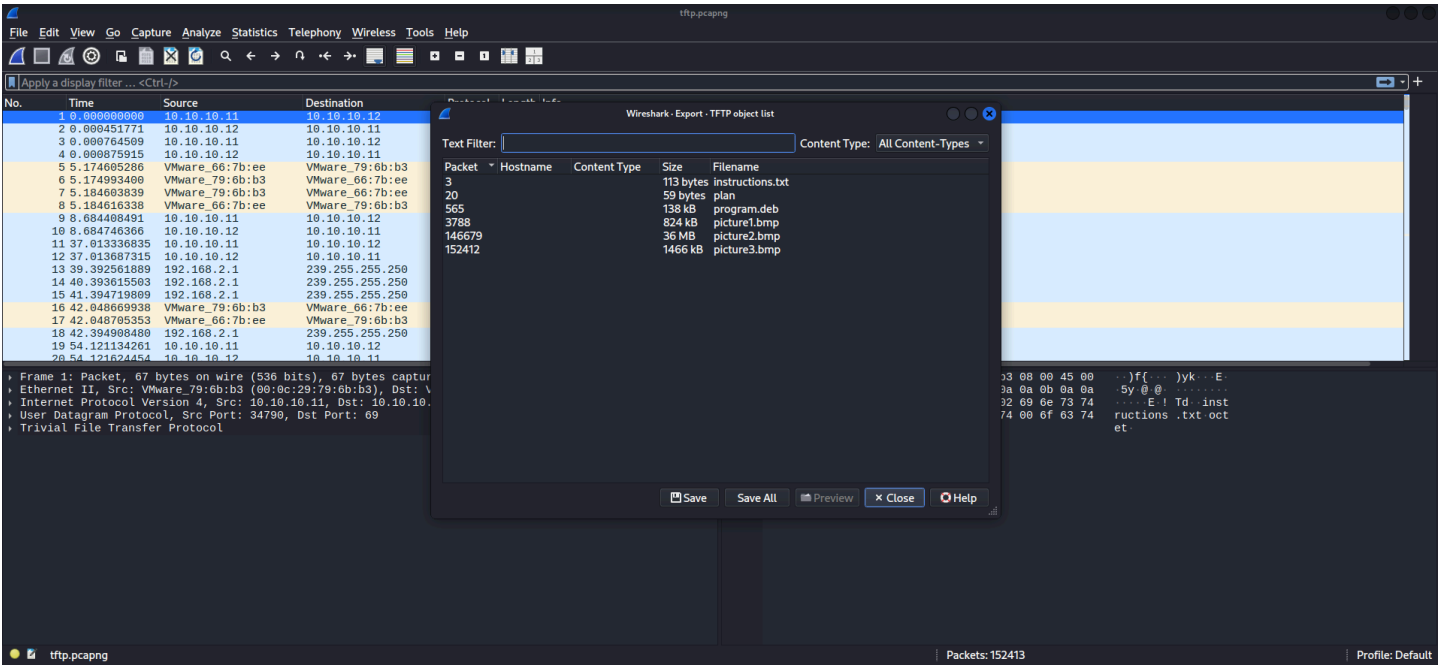
The protocol is **TFTP (Trivial File Transfer Protocol)**. Wireshark can extract transferred files automatically.

Extract TFTP objects:

1. Go to **File** → **Export Objects** → **TFTP**



2. Select all files and save them to your current directory



Extracted files:

instructions.txt picture1.bmp picture2.bmp picture3.bmp plan program.deb

Step 6: Analyze the Text Files

cat instructions.txt

Output:

GSQCQBRFAGRAPELCGBHEGENSSVPFBJRZHFGQVFTHVFRBHESYNTGENAFSRE.SVTERBHGNJNLGBUVQRGURSNTN
AQVJVYYYPURPXONPXSBEGURCYNA

This looks like **ROT13** (a Caesar cipher shifting letters by 13 positions).

Decode it:

echo
"GSQCQBRFAGRAPELCGBHEGENSSVPFBJRZHFGQVFTHVFRBHESYNTGENAFSRE.SVTERBHGNJNLGBUVQRGURSNTN
NAQVJVYYYPURPXONPXSBEGURCYNA" | tr 'A-Za-z' 'N-ZA-Mn-za-m'

Output:

TFTPDOSNTENCRYPTOURTRAFFICSOWEMUSTDISGUISEOURFLAGTRANSFER.FIGUREOUTAWAYTOHIDETHEFLAGA
NDIWILLCHECKBACKFORTHEPLAN

Add spaces for readability:

TFTP DOESNT ENCRYPT OUR TRAFFIC SO WE MUST DISGUISE OUR FLAG TRANSFER. FIGURE OUT A
WAY TO HIDE THE FLAG AND I WILL CHECK BACK FOR THE PLAN

Now check the `plan` file:

```
cat plan
```

Output:

```
VHFRQGURCEBTENZNAQUVQVGJVGU-QHRQVYVTRAPR.PURPXBHGGURCUBGBF
```

Decode with ROT13:

```
echo "VHFRQGURCEBTENZNAQUVQVGJVGU-QHRQVYVTRAPR.PURPXBHGGURCUBGBF" | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

Output:

```
IUSEDTHEPROGRAMANDHIDITWITH-DUEDILIGENCE.CHECKOUTTHEPHOTOS
```

Add spaces:

```
I USED THE PROGRAM AND HID IT WITH-DUEDILIGENCE. CHECKOUT THE PHOTOS
```

```
file program.deb
```

Output:

```
program.deb: Debian binary package (format 2.0), with control.tar.gz , data  
compression xz
```

Key finding: The program used is `steghide` (a steganography tool) and the password is hinted as `DUEDILIGENCE` .

Step 7: Analyze the BMP Images

Check the image file types:

```
file picture*
```

Output:

```
picture1.bmp: PC bitmap, Windows 3.x format, 605 x 454 x 24, image size 824464,
resolution 5669 x 5669 px/m, cbSize 824518, bits offset 54
picture2.bmp: PC bitmap, Windows 3.x format, 4032 x 3024 x 24, image size 36578304,
resolution 5669 x 5669 px/m, cbSize 36578358, bits offset 54
picture3.bmp: PC bitmap, Windows 3.x format, 807 x 605 x 24, image size 1466520,
resolution 5669 x 5669 px/m, cbSize 1466574, bits offset 54
```

Search for visible text in the images:

```
strings picture* | grep -i "picoCTF"
```

Output: (no results)

Check for steganography with `zsteg`:

```
zsteg picture3.bmp
```

Output (excerpt):

```
[.] picture3.bmp
imagedata          .. text: "3_PHyg2bN!M<1YJ2]Nd"
b1,b,lsb,xY        .. file: OpenPGP Public Key
b1,b,msb,xY        .. text: "bUqjc(/=mA"
b2,g,msb,xY        .. file: OpenPGP Public Key
```

Interesting – we see references to “OpenPGP Public Key”. This suggests data is hidden.

Step 8: Extract Hidden Data with Steghide

Use `steghide` on `picture3.bmp`:

```
steghide info picture3.bmp
```

Output:

```
"picture3.bmp":  
  format: Windows 3.x bitmap  
  capacity: 59.6 KB  
Try to get information about embedded data ? (y/n) y  
Enter passphrase:
```

The password is `DUEDILIGENCE` (from the decoded `plan` file).

```
steghide extract -sf picture3.bmp -p DUEDILIGENCE
```

Output:

```
wrote extracted data to "flag.txt".
```

Step 9: Read the Flag

```
cat flag.txt
```

Output:

```
picoCTF{<REDACTED>}
```

Final Result

```
picoCTF{<REDACTED>}
```

Note: The actual flag is redacted here. In the real challenge, following these steps will reveal the complete flag.

Key Concepts

Concept	Description
TFTP (Trivial File Transfer Protocol)	A simple file transfer protocol often used in CTF challenges to transfer files without encryption.
ROT13 Cipher	A simple substitution cipher shifting letters by 13 positions (self-inverse).
Steganography	Hiding data within image files (BMP, PNG, JPEG) without visible changes.
Steghide	A command-line tool for embedding and extracting hidden data in BMP, JPEG, WAV, and AU files.
Password Hints	Decoded messages often contain passwords or instructions for further steps.

Pro Tips

1. **Use Wireshark’s “Export Objects” feature** – This automatically extracts files transferred over TFTP, HTTP, SMB, and other protocols.
 2. **ROT13 is recognizable** – If you see text with only uppercase letters and no spaces, try ROT13 decoding.
 3. `tr 'A-Za-z' 'N-ZA-Mn-a-m'` – This is the quickest way to decode ROT13 on the command line.
 4. **Steghide is common for BMP files** – When you see BMP images in a PCAP, suspect steganography.
 5. **Check image metadata with `zsteg`** – This tool can detect LSB steganography in PNG/BMP files and may reveal clues.
 6. **Read the decoded messages carefully** – They often contain instructions (e.g., “checkout the photos”, “used the program”).
-

Alternative ROT13 Decoding Tools

Tool	Command / Link
Linux <code>tr</code>	<code>tr 'A-Za-z' 'N-ZA-Mn-za-m'</code>
CyberChef	https://gchq.github.io/CyberChef/ (Operation: ROT13)
Python	<code>import codecs; codecs.decode('text', 'rot_13')</code>
Online	https://rot13.com/

Alternative Steganography Tools

Tool	Purpose
<code>steghide</code>	Extract embedded data from BMP, JPEG, WAV, AU
<code>zsteg</code>	Detect LSB steganography in PNG/BMP
<code>stegsolve</code>	GUI tool for analyzing bit planes
<code>binwalk</code>	Detect embedded files within other files